Amune Personal Finance Tracker — Project Overview

Executive summary

Amune is a full-stack personal finance tracker that records income and expenses, categorizes each one automatically with AI, and shows the results in a dashboard in Indian Rupees (₹). It was built end to end: a FastAPI backend, a React frontend, a SQLite database with versioned migrations, and an auto-generated Python SDK.

Three things set it apart from a basic CRUD app:

- **Business-rule enforcement.** The backend rejects logically wrong entries (such as "Rent" logged as income) and blocks accidental duplicates within a 5-minute window.
- **AI with a safety net.** Categories come from a Groq-hosted language model, and a keyword dictionary takes over whenever the model is unavailable, so the system never fails because of the AI.
- **Production-style engineering.** Layered architecture, database-level constraints, schema migrations, an OpenAPI-generated SDK, one-command setup scripts, and 42 automated tests.

This document explains the project from scratch: what it does, how it is built, and why each design decision was made.

Problem and goals

People lose track of where their money goes because logging transactions is tedious and categorizing them is worse. Amune removes the categorization step and guards data quality, so the numbers a user sees can be trusted.

Goals

1. Let a user add a transaction with only a description, an amount, and a type (income or expense).
2. Categorize it automatically, while letting the user correct the category later.
3. Reject bad data: contradictory entries, accidental double-submits, and invalid amounts.
4. Present totals, category breakdowns, and monthly trends clearly in ₹.

5. Expose a clean, documented API and a generated SDK so other programs can use it.
6. Make setup and verification simple for any reviewer, with no manual configuration.

Technology stack

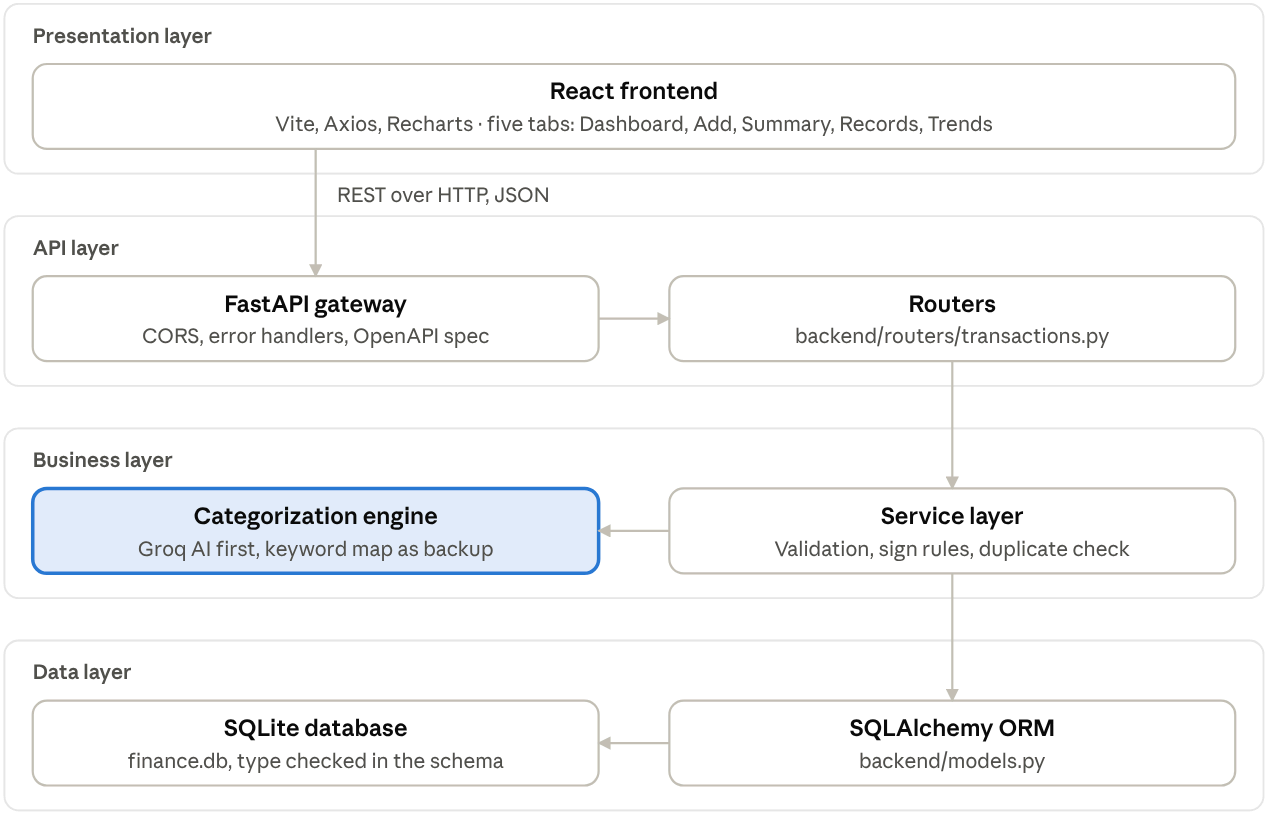
Each layer uses a mainstream, well-supported tool chosen for a specific job.

Layer	Technology	Role in the project
Frontend	React 18, Vite, Axios, Recharts	Dashboard UI, forms, charts, HTTP calls
Backend	FastAPI, Pydantic	REST API, request validation, error mapping
Business logic	Python service layer	Validation rules, duplicate checks, sign handling
AI	Groq Cloud API, model openai/gpt-oss-20b	Primary transaction categorization
Fallback	Hard-coded keyword dictionary	Categorization when the AI is unavailable
Database	SQLite (finance.db), SQLAlchemy ORM	Persistent storage
Migrations	Alembic	Versioned schema changes
SDK	OpenAPI Generator CLI	Auto-generated Python client
Testing	pytest	42 automated tests
Tooling	Windows batch scripts	One-command setup and launch

System architecture

Amune follows Clean Architecture: each layer talks only to its neighbour, so swapping the AI model or the database does not ripple through the rest of the code.

Four layers separate the screen, the API, the rules and the data

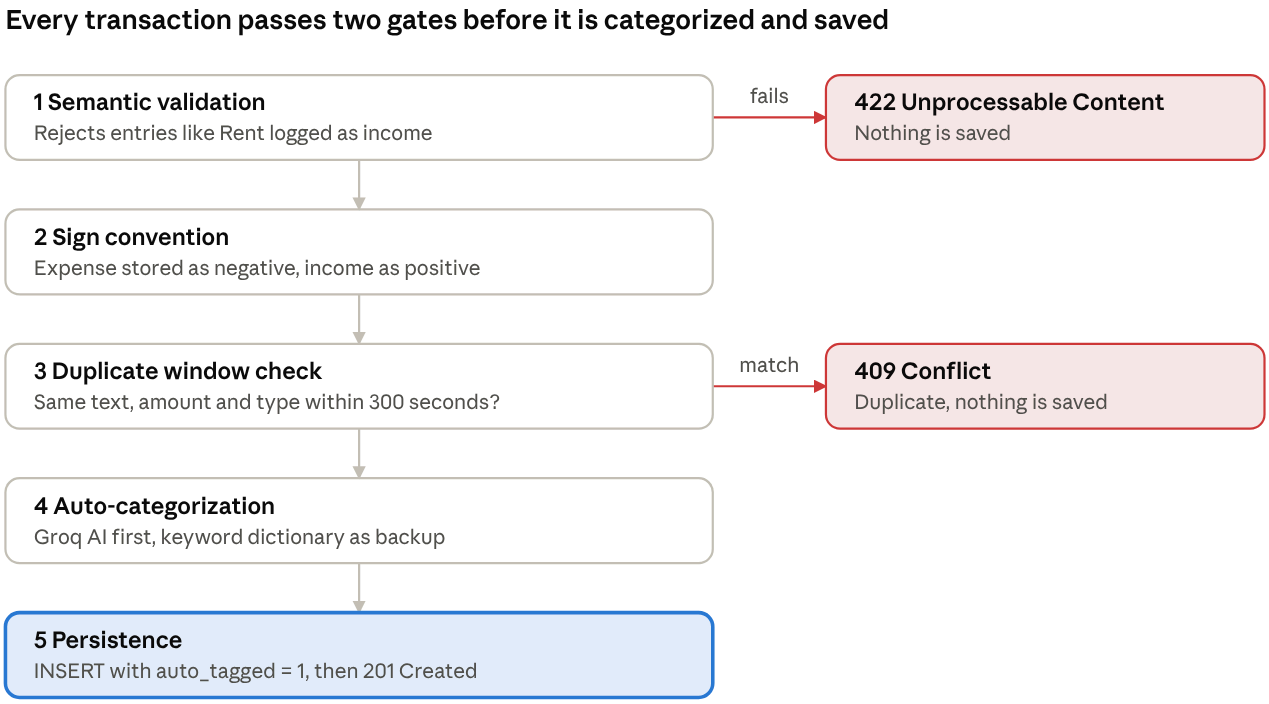


architecture · 4 layers, 7 components

A request moves along the arrows: the React client calls the FastAPI gateway, the router hands it to the service, and the service uses the categorization engine and the ORM. Alembic versions the SQLite schema, and the OpenAPI spec that FastAPI publishes is the source the Python SDK is generated from.

Transaction creation pipeline

A request to `POST /transactions/` runs through five stages in order, and stages 1 and 3 can stop it before anything is written.



POST /transactions/ pipeline · 5 stages, 2 rejection exits

The client sends a description, a strictly positive amount, and a type (income or expense). A rejected request returns an error and leaves the database untouched. An accepted request returns `201 Created` with the amount shown as a positive number, whatever sign is stored.

Database design and migrations

All data lives in one SQLite table, `transactions`, in the file `finance.db`. Key rules are enforced by the database itself, not only by application code.

Column	Type	Constraints	Purpose
id	INTEGER	Primary key, autoincrement	Unique transaction identifier
description	TEXT	NOT NULL	What the user typed
amount	REAL	NOT NULL	Negative for expenses, positive for income
transaction_type	TEXT	NOT NULL, CHECK in ('income', 'expense')	Type, enforced at the database level

Column	Type	Constraints	Purpose
category	TEXT	Nullable	Food, Transport, and so on
auto_tagged	BOOLEAN	NOT NULL, default 1	1 if set automatically, 0 if the user changed it
timestamp	TIMESTAMP	Default CURRENT_TIMESTAMP	UTC creation time

Migrations. The schema is created and versioned with Alembic (migration `0a913ffcb657_create_transactions_table`). Running `alembic upgrade head` builds the database from nothing, and the setup script runs it automatically. This means any reviewer gets the exact same schema, and future changes can be applied safely without deleting data.

Business rules

Four rules protect data quality. They live in the service layer, so they apply equally to the web app, the API, and the SDK.

1. Sign convention

Clients always send a positive amount (validated as greater than zero). The database stores expenses as negative numbers and income as positive ones, which makes totals a simple sum. On the way out, the API returns absolute values again, so the UI and SDK users never have to interpret signs.

2. Semantic type validation

Before saving, the service checks whether the description contradicts the chosen type. A conflict returns `422 Unprocessable Content`.

Type chosen	Keywords that trigger a rejection	Example
Income	rent, mortgage, groceries, grocery, uber, ola, taxi, netflix, spotify, electricity, fuel, petrol, restaurant, coffee, pharmacy	"Monthly Apartment Rent" as income is rejected
Expense	salary, paycheck, payroll, dividend, bonus, stipend, wages	"Company Salary" as expense is rejected

3. Duplicate detection

A sliding window of 300 seconds (configurable through `DUPLICATE_WINDOW_SECONDS`) catches accidental double submissions. A new entry is a duplicate only if all of these match an entry inside the window:

- the same transaction type
- the same amount, to within 0.0001
- the same description after normalization

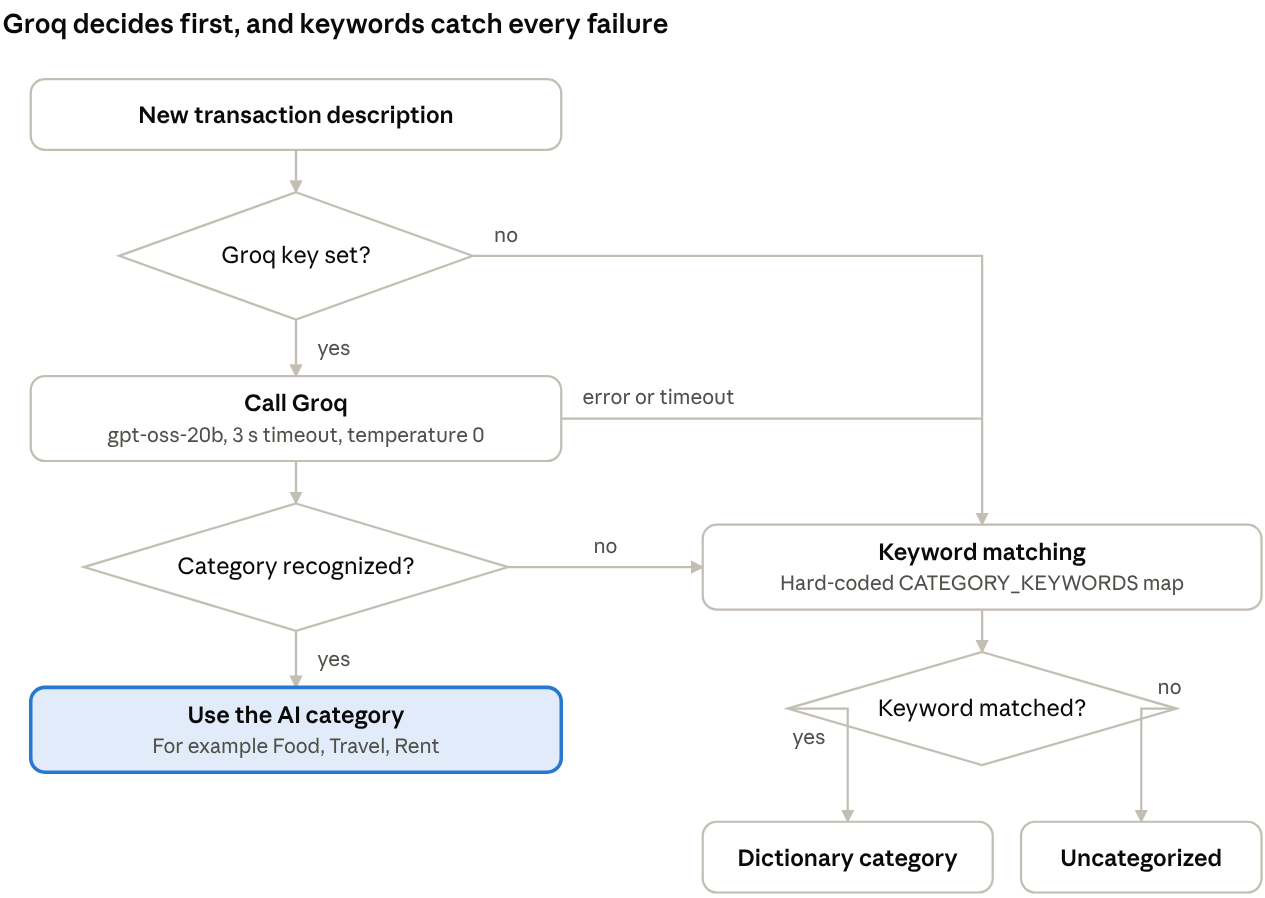
A duplicate returns `409 Conflict`. The same entry sent after the window has passed is allowed, since a user may genuinely repeat a purchase.

4. Category reclassification

`PATCH /transactions/{id}/reclassify` lets a user correct a category. The category is updated and `auto_tagged` is set to 0 permanently. This keeps an audit trail of human corrections and ensures they are never overwritten by automatic categorization.

AI auto-categorization

Each new transaction gets a category with no user input. The engine asks a Groq-hosted language model (`openai/gpt-oss-20b`, temperature 0.0 for repeatable answers, 3-second timeout) and accepts the answer only if it is a recognized category.



categorization decision flow · 3 decisions, 3 outcomes

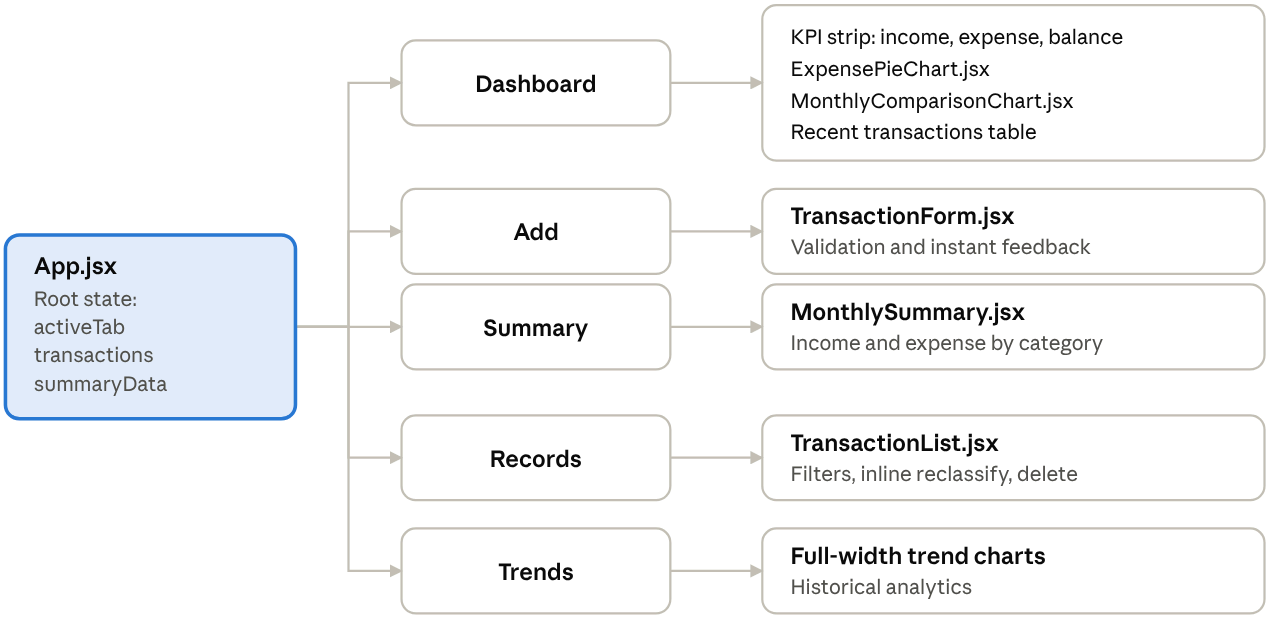
If no API key is configured, if Groq errors or times out, or if its answer is not a known category, the engine falls back to a hard-coded keyword dictionary. If nothing matches there either, the category is "Uncategorized". The user is never blocked by an AI outage.

Security. The Groq key lives in a `.env` file that is listed in `.gitignore` and never committed. A reviewer can add their own key or run the system on the keyword dictionary alone.

Frontend design

The web client is a React 18 single-page app built with Vite, using Axios for API calls and Recharts for charts. A navigation bar switches between five tabs, which keeps each screen short instead of one long scrolling page.

One root component feeds five tabs, and each tab owns its views



frontend component tree · 1 root, 5 tabs

`App.jsx` owns the shared state (active tab, transactions, summary data) and passes it down. Notable behaviors:

- **Rupee formatting.** Every amount, including KPI cards, tables, forms and chart tooltips, is shown in ₹.
- **Balanced dashboard.** The pie chart and the monthly comparison chart each take 50% of the width.
- **Live sync.** Adding, reclassifying or deleting a transaction reloads data in every component straight away.
- **Inline editing.** The Records tab filters by month, type and category, and lets the user fix a category or delete a row in place.

Python SDK

The `finance_sdk/` package is generated, not hand-written, from the API's OpenAPI contract. When the backend changes, the SDK is regenerated and stays in sync automatically.

```
openapi-generator-cli generate -i http://localhost:8000/openapi.json -g
python -o finance_sdk
```

Using it takes a few lines:

```
from finance_sdk import ApiClient
from finance_sdk.api.transactions_api import TransactionsApi
from finance_sdk.models.transaction_create import TransactionCreate

api = TransactionsApi(ApiClient())

# Add a transaction
new_tx = api.create_transaction(
    transaction_create=TransactionCreate(
        description="Dinner at Italian Bistro",
        amount=45.0,
        transaction_type="expense"))

# Monthly summary
summaries = api.get_summary(month="2026-10")

# Correct the category
updated = api.reclassify_transaction(
    transaction_id=new_tx.id,
    reclassify_request={"category": "Food"})
```

The script `sdk_smoke_test.py` exercises all five endpoints through the SDK against a live server and prints `All SDK smoke test assertions passed successfully!` on success.

Developer experience and testing

A reviewer can go from a clean clone to a running app with two scripts and no manual configuration.

Script	What it does
`setupdev.bat`	Creates the Python environment, installs `requirements.txt`, copies `.env.example` to `.env` if no `.env` exists, runs `alembic upgrade head`, and runs `npm install` in `frontend/`
`runapplication.bat`	Starts the FastAPI backend on port 8000 and the Vite frontend on port 3000, each in its own window
`pytest -v`	Runs all 42 automated tests

Script	What it does
`python sdk_smoke_test.py`	Checks all five endpoints end to end through the SDK

The 42 tests cover creating, reading and deleting transactions, duplicate detection, the sign convention, AI categorization (including the fallback path), and the SDK.

Engineering decisions

The choices below show how the project was designed for correctness, resilience and ease of review, not only for working features.

Decision	Reason	What it demonstrates
Layered architecture	Keeps UI, API, rules and data independent, so each can change on its own	Maintainable system design
Signed amounts in storage, absolute amounts in the API	Totals become simple sums while clients see consistent positive numbers	API design that hides internal detail
CHECK constraint on transaction type	The database rejects invalid types even if application code is bypassed	Defense in depth
Semantic and duplicate validation	Catches logical errors and double-clicks before they corrupt totals	Attention to data integrity
AI with a keyword fallback	Gives smart categories without making the AI a single point of failure	Resilient integration of a third-party service
`auto_tagged` flag	Records which categories a human corrected and protects them from being overwritten	Auditability

Decision	Reason	What it demonstrates
Alembic migrations	Makes the schema reproducible and safely changeable	Database discipline
Generated SDK from OpenAPI	The client always matches the live API contract	Platform thinking
Setup scripts and 42 tests	Anyone can install, run and verify the project in minutes	Delivery and quality focus

How to run and evaluate it

1. Clone the repository and run `setupdev.bat`. It prepares the environment, the `.env` file, the database and the frontend dependencies.
2. Optional: add a `GROQ_API_KEY` to `.env` to enable AI categorization. Without it, the keyword dictionary is used.
3. Run `runapplication.bat`, then open the frontend at port 3000. The API runs on port 8000, and its interactive documentation is generated from `/openapi.json`.
4. Add a transaction such as "Dinner at Italian Bistro" and watch it appear in the Records tab and on the Dashboard with an automatic category.
5. Try the rules: log "Monthly Apartment Rent" as income (rejected with 422), or submit the same entry twice within five minutes (rejected with 409).
6. Run `pytest -v` for the 42 tests, and `python sdk_smoke_test.py` with the server running to verify the SDK.